Algoritmos Distribuidos
Abrazo Mortal

Claudia León

Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

Octubre 2008 – Marzo 2009



Objetivos

- Comprender y describir el problema de abrazo mortal distribuido
- Diferenciar el abrazo mortal de recursos y de comunicación
- Comprender y comparar diferentes enfoques para manejar el problema de abrazo mortal
- Comparar diferentes algoritmos para detectar abrazo mortal.

Qué es un abrazo mortal ?

Abrazo mortal es una situación en la que se encuentran un conjunto de procesos donde cada proceso está esperando por un evento que sólo puede ser producido por otro proceso perteneciente a ese mismo conjunto.

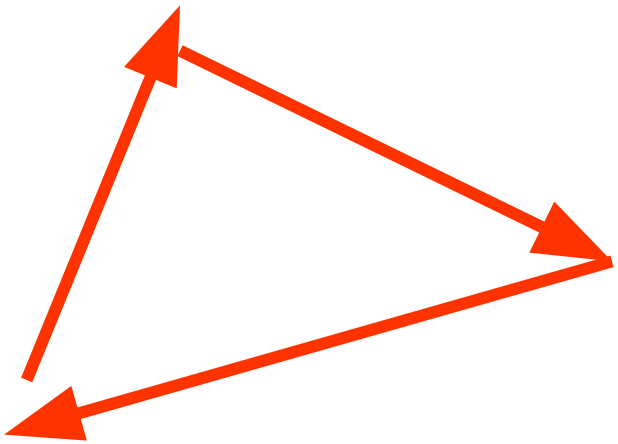
Tipos de abrazo mortal

1. **Abrazo mortal por recursos**: Un conjunto de procesos se encuentran en abrazo mortal por recursos si cada proceso perteneciente al conjunto requiere recursos que posee otro proceso perteneciente al mismo conjunto y requiere obtener todos los recursos para poder ser desbloqueado.

2. **Abrazo mortal por comunicación**: Un conjunto de procesos está en abrazo mortal por comunicación si cada proceso en el conjunto está esperando para comunicarse con otro proceso en el mismo conjunto, y ningún proceso puede iniciar una comunicación hasta tanto no reciba la comunicación por la cual está esperando.

Tipos de abrazo mortal

El abrazo mortal por comunicación es propio de sistemas distribuidos, por tanto será el principal objeto de estudio en este curso.



Grafo de espera o *wait-for-graph* o (WFG) o *transaction-wait-for graph (TWF graph)*

Existe un arco de P_i a P_j si P_i está bloqueado esperando por un mensaje de P_j

Estrategias para manejo de abrazo mortal en S.D.

- **Prevención de deadlock**: Significa asegurar que un proceso posea todos los recursos antes de su ejecución. No aplica en deadlock por comunicación. Puede ser impráctico para recursos.
- **Detección deadlock**: Consiste en realizar verificaciones periódicas para detectar la presencia o no de deadlock: esto es la presencia o ausencia de un ciclo en el grafo de espera.

Estrategias para manejo de abrazo mortal en S.D.

En sistemas distribuidos la **detección** de deadlock es la estrategia favorable debido a:

- Una vez que se presenta un ciclo en el WFG, va a persistir hasta que sea detectado y roto.
- La detección de ciclos puede realizarse simultáneamente con la aplicación, y por lo tanto no afecta el rendimiento de la aplicación.

Por estas razones, las soluciones propuestas en la literatura se enfocan mayoritariamente en la estrategia de detección de deadlock.

Detección de abrazo mortal

Se distinguen dos etapas o actividades:

- **Detección de abrazo mortal** propiamente dicha. Envuelve a su vez dos aspecto:
  1. Mantenimiento del WFG
  2. Búsqueda de circuitos en el WFG

- **Resolución del abrazo mortal**: Implica romper dependencias de espera en el WFG. Esto implica cancelar uno o más de los procesos involucrados en el deadlock a fin de permitir el progreso de otros.

Detección de abrazo mortal

Un algoritmo CORRECTO para la detección de abrazo mortal debe satisfacer las siguientes condiciones:

1. Si existe un abrazo mortal el algoritmo debe detectarlo en un tiempo finito (Algoritmo confiable)
2. El algoritmo no debe reportar deadlock que no existen, esto es no debe detectar falsos deadlocks (Algoritmo seguro).

Detección de abrazo mortal

En sistemas distribuidos es complejo diseñar un algoritmo correcto para la detección de abrazo mortal debido a que los nodos pueden obtener los WFG fuera de tiempo o inconsistentes.

Por esta razón los nodos pueden detectar un ciclo que no existe, porque los diferentes segmentos del grafo existieron en el sistema en tiempos distintos.

Esta es la principal causa por la que algunos algoritmos publicados y en uso son incorrectos: detectan falsos deadlocks

Organización del control en la detección de abrazo mortal

- **Control Centralizado**: Un nodo designado tiene la responsabilidad de construir el WFG global y detectar los ciclos
- **Control Distribuido**: La responsabilidad de detectar un deadlock global es compartida equitativamente entre todos los nodos
- **Control Jerárquico**: Los nodos se organizan en forma jerárquica y cada nodo detecta deadlocks en los que estén involucrados sus descendientes.

Control centralizado

1. El nodo de control puede mantener el WFG global constantemente o lo puede construir en un momento dado
2. Son conceptualmente simples y fácil de implementar
3. La resolución del deadlock es óptima ya que el nodo de control tiene toda la información y puede tomar la decisión más acertada.

Control centralizado: problemas

1. El embotellamiento que se genera alrededor del nodo de control.
2. El tráfico de mensajes para detectar deadlock es independiente de la estructura de los ciclos que se formen o la frecuencia de la presencia de deadlock. Es decir, hayan o no deadlocks el nodo registra toda la actividad

Control distribuido

1. Cada nodo mantiene una porción del WFG global y todos los nodos cooperan de forma colectiva para detectar un ciclo en el grafo global.
2. Cualquier nodo puede iniciar el proceso, lo cual los hace más resistentes a fallas y ningún nodo está sobrecargado con la detección de deadlock

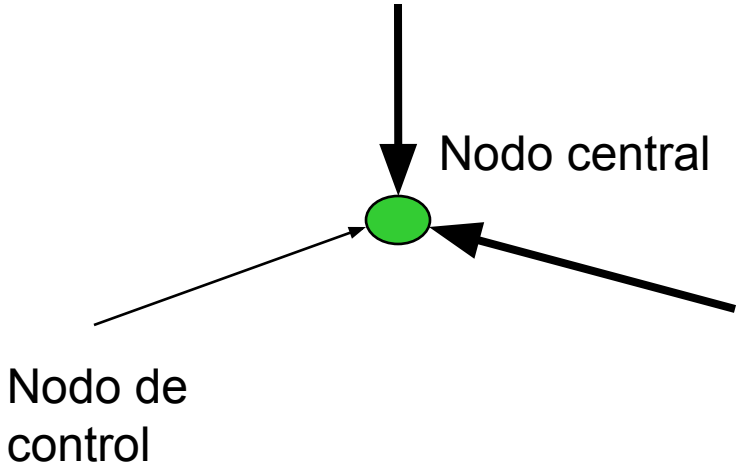
Control distribuido: problemas

1. Debido a la falta de memoria compartida su diseño resulta muy complejo
2. Los nodos pueden reportar falsos deadlock en base a segmentos del WFG que nunca existieron simultáneamente.

Control jerárquico

1. Es el intermedio entre el centralizado y el distribuido
2. Explota los patrones de comunicación locales a un grupo de nodos para detectar el deadlock
3. No es dependiente de la falla de un nodo
4. Un nodo no está sobrecargado por la detección de deadlock en los cuales muy posiblemente no está involucrado.

Control jerárquico



Clasificación de algoritmos distribuidos para detección de deadlock

- **Path-pushing**: La información referente a dependencias wait-for son difundidas en forma de caminos (secuencias de arcos de dependencias). Obermarck.
- **Edge-chasing**: Un proceso bloqueado hace circular un mensaje especial llamado prueba a través del grafo de dependencias y declarará deadlock cuando reciba la prueba iniciada por el mismo. Las pruebas son generalmente de tamaño fijo y pequeño. Chandy y otros.
- **Difusión de cálculo**: Un proceso difunde una consulta a través de sus arcos de salida en WFG, estas consultas son propagadas por los procesos bloqueados. Cuando recibe toda las respuestas de todas sus consultas, declara deadlock. Chandy y otros.
- **Estado Global**: Toman un estado global del sistema y lo examinan para detectar deadlock.

Algoritmos para la detección de abrazo mortal

- Algoritmos de Chandy, Misra y Haas
  - De Edge-Chasing (1)
  - De Difusión de Cálculo (2)
- Algoritmo de Obermarck

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

La idea del algoritmo es detectar un ciclo formado por los procesos que se encuentran interbloqueados.

Por ser un algoritmo de control distribuido, los procesos pueden estar ubicados en distintos nodos, es decir, el algoritmo puede realizar detección de abrazos mortales remotos.

El envío de mensajes entre los procesos del sistema es fiable, es decir, no hay pérdida de mensajes. La comunicación no es necesariamente síncrona.

Los procesos y las dependencias locales se conocen.

Cada proceso trata de conocer su conjunto dependiente: El conjunto dependiente de un proceso P está formado por todos aquellos procesos que posean directa o transitivamente algún recurso que P necesita para proseguir con su ejecución.

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Se tiene para cada proceso P_i un arreglo lógico $Dependiente_i[j]$

$Dependiente_i[j]$ = Verdad $\Leftrightarrow$ P_i sabe que P_j pertenece a su conjunto dependiente.

Ejemplo:

Conjuntos dependientes:

P1: {P2, P3}

P2: {P3}

P3: {P4}

P4: {P5,P6,P7}

P5: {P6,P7}

P6: {P8}

P7: {P10}

P8: {P9}

P9: {P1}

P10: { }

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Se tiene para cada proceso P_i un arreglo lógico Local[j]
$Local_i[j]$ = Verdad $\Leftrightarrow$ P_i y P_j pertenecen al mismo nodo.
Ejemplo:

Localidad:

P1: {P2, P3}

P2: {P1,P3}

P3: {P1,P2}

P4: {P5,P6,P7}

P5: {P4,P6,P7}

P6: {P5,P6,P7}

P7: {P4,P5,P6}

P8: {P9,P10}

P9: {P8,P10}

P10: {P8,P9}

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Estados de un Proceso.

- **Estado Bloqueado:** un proceso está en estado **Bloqueado** si no está ejecutando alguna tarea debido a su espera por uno o más recursos.
- **Estado En Ejecución:** un proceso está en estado **En Ejecución** si no está esperando por algún recurso, es decir, su conjunto dependiente es vacío.

Los procesos en estado **Bloqueado** no pueden enviar mensajes inherentes a la aplicación que se esté ejecutando en ese sistema; solo podrán enviar mensajes correspondientes al algoritmo para detectar el abrazo mortal.

Un proceso en estado **Bloqueado** pasa al estado **En Ejecución** si al menos uno de los procesos contenidos en su conjunto dependiente le envía un mensaje de aplicación (el proceso **Bloqueado** despertado).

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Una consulta (o prueba) para detectar el abrazo mortal es iniciada por cualquier proceso en estado bloqueado, ya que éste se encuentra demasiado tiempo inactivo y sospecha de la presencia de un abrazo mortal. A este proceso se le llama **proceso iniciador**.

Una consulta es una tripleta: (i,j,k) que indica que corresponde a la detección de deadlock iniciada por el Proceso Pi enviada desde el proceso Pj al Proceso Pk

Una consulta (o prueba) viaja a través de los arcos del WFG global, y un abrazo mortal es detectado cuando la prueba retorna a su proceso iniciador.

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Para iniciar el Algoritmo por Pi.

Si dependiente$_i$[i] entonces **"Declarar deadlock"**
sino Paratodo {j,k / local[j] and ¬local[k], dependiente$_i$[j] and dependiente$_j$[k]} hacer
enviar(prueba,i, j, k) a k.
Fparatodo
fsi

Cuando P_m reciba (Prueba, i, j, m)

Si Estado(P_m) = *Bloqueado* and ¬dependiente$_m$[i] entonces
dependiente$_m$[i]) = verdad
Si m = i entonces **"Declarar deadlock"**
sino Paratodo {j,k / local[j] and ¬local[k], dependiente$_m$[j] and dependiente$_m$[k]} hacer
enviar(prueba,i, j, k) a k.
Fparatodo
Fsi
Fsi

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Conjuntos dependientes:

P1: {P2, P3}

P2: {P3}

P3: {P4}

P4: {P5,P6,P7}

P5: {P6,P7}

P6: {P8}

P7: {P10}

P8: {P9}

P9: {P1}

P10: { }

(1,3,4)

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Conjuntos dependientes:

- P1: {P2, P3}
- P2: {P3}
- P3: {P4}
- P4: {P5,P6,P7}
- P5: {P6,P7}
- P6: {P8}
- P7: {P10}
- P8: {P9}
- P9: {P1}
- P10: { }

(1,3,4)

(1,6,8)

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Conjuntos dependientes:

- P1: {P2, P3}
- P2: {P3}
- P3: {P4}
- P4: {P5,P6,P7}
- P5: {P6,P7}
- P6: {P8}
- P7: {P10}
- P8: {P9}
- P9: {P1}
- P10: { }

(1,3,4)

(1,6,8)

(1,7,10)

ALGORITMOS DE CHANDY-MISRA-HAAS (1)

Conjuntos dependientes:

P1: {P2, P3}

P2: {P3}

P3: {P4}

P4: {P5,P6,P7}

P5: {P6,P7}

P6: {P8}

P7: {P10}

P8: {P9}

P9: {P1}

P10: { }

(1,9,1)

(1,6,8)

(1,3,4)

(1,7,10)

ALGORITMOS DE CHANDY-MISRA-HAAS (1)



(1,9,1)

(1,6,8)

(1,3,4)

(1,7,10)

Conjuntos dependientes:

P1: {P2, P3}

P2: {P3}

P3: {P4}

P4: {P5,P6,P7}

P5: {P6,P7}

P6: {P8}

P7: {P10}

P8: {P9}

P9: {P1}

P10: { }

ALGORITMOS DE CHANDY-MISRA-HAAS (1): Conclusiones

- El algoritmo detecta todos los abrazos mortales que estén presentes en el sistema.
- El algoritmo no detecta falsos deadlock. Todos los abrazos mortales detectados son genuinos.
- El orden del algoritmo es de m(n-1)/2 para detectar un abrazo mortal que involucra m procesos dispersos en n nodos (sólo envía un mensaje o prueba por cada arco en el WFG que relaciona procesos en nodos diferentes.
- El tamaño de los mensajes en fijo y pequeño: tres enteros.
- Como el algoritmo no almacena el ciclo de procesos que intervienen en el abrazo mortal, entonces una de las alternativas para resolver el abrazo mortal es matando al proceso iniciador de la consulta.



ALGORITMOS DE CHANDY-MISRA-HAAS (2). Actividad

Edo. Inicial del Sistema

P1: Bloqueado, esperando por el proceso P2 ó por el proceso P3.

P2: Bloqueado, esperando por el proceso P4.

P3: Bloqueado, esperando por el proceso P1 ó por el proceso P4.

P4: En Ejecución (está activo)

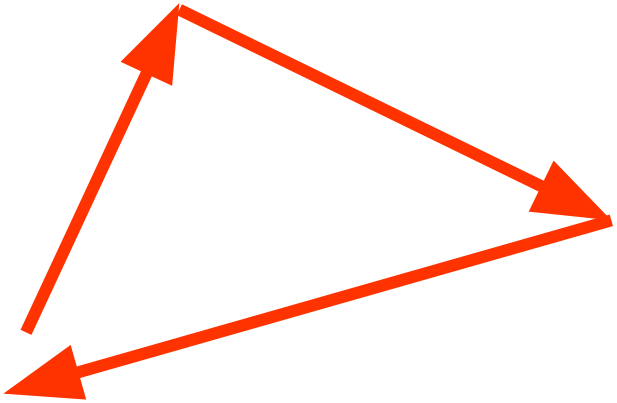
Resolución de deadlock

La persistencia de un abrazo mortal tiene dos efectos negativos sobre el rendimiento de un sistema:

- Los recursos en manos de procesos interbloqueados no están disponibles a otros procesos.
- El tiempo mientras persista el deadlock se suma al tiempo de respuesta de cada proceso interbloqueado.

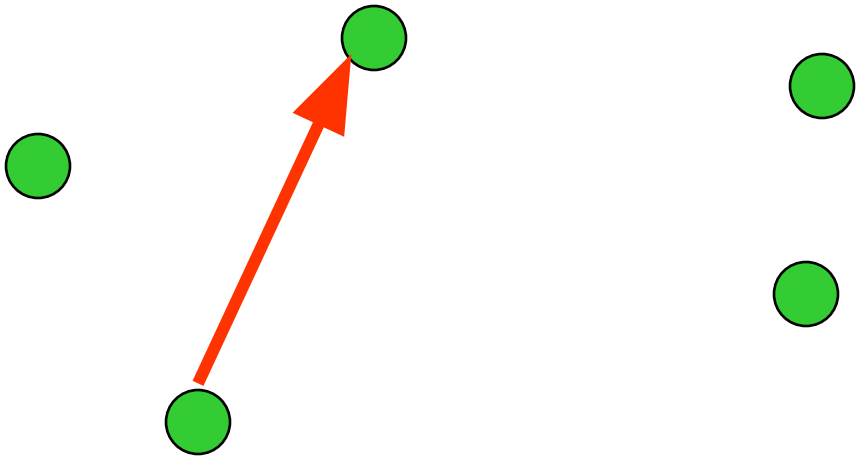
Es necesario resolver de forma rápida y eficiente un deadlock ya detectado.

Resolución de deadlock



1. Abandonando al menos uno de los procesos involucrados en el deadlock
2. Garantizando la asignación de los recursos liberados a otros procesos involucrados en el deadlock.

Resolución de deadlock



1. La eficacia de la resolución depende de la información que se tenga acerca de los procesos interbloquados y los recursos que cada uno mantiene.

Problemas en la resolución de deadlock

1. Un proceso que detecta un deadlock no conoce todos los procesos involucrados
2. Dos o más procesos pueden detectar independientemente el mismo deadlock

Prioridad única a cada proceso (Obermarck)

1. Cada deadlock es detectado sólo por el proceso de mayor prioridad involucrado en dicho deadlock,
2. Cuando el proceso de mayor prioridad detecta un deadlock, conoce cual es el proceso de menor prioridad en el ciclo, el cual será abandonado.

No siempre abandonar al de menor prioridad resulta la solución más eficiente.

Pasos para la resolución de deadlock

1. La “víctima” debe ser abandonada, los recursos que mantenía liberados, el estado de estos recursos restaurados a su estado previo (Ref: Edos Locales) debe asegurarse la asignación a los procesos bloqueados. Si un proceso puede esperar por muchos recursos, se puede originar un nuevo deadlock.
2. La información del deadlock detectado que concierne a la víctima debe ser eliminada lo más pronto posible en todos los nodos. Sino puede ser contada en otros ciclos y detectar falsos deadlock.

Detección y resolución de deadlock

1. La espera por recursos agrega arcos y nodos al WFG
2. La resolución de deadlock remueve arcos y nodos del WFG

La integración de estas dos actividades debe realizarse cuidadosamente para evitar la detección de falsos deadlock.